



HEALTHCARE SERVICES PRIVATE LIMITED

Information Security Compliance ISO 27001 — Security Questionnaire

Official Responses to Information Security Audit Queries

Document Type: ISO 27001 Security Compliance Response

Prepared By: MedNu Healthcare Services Pvt. Ltd.

Platform: MedNu Patient App & MedNu Doctor App

Technology Stack: Flutter · Firebase · Google Cloud Platform

Date: June 17, 2026

Version: 1.0

Overview — Security Controls Summary

Query	Topic	Status
Query 1	Internal Data Security	IMPLEMENTED
Query 2	Data Breach & Cyber Fraud Controls	IMPLEMENTED
Query 3	Cloud Services Information Security	IMPLEMENTED
Query 4	Access to Source Code	IMPLEMENTED
Query 5	Data Storage, Retention & Deletion	IMPLEMENTED
Query 6	Web Filtering	IMPLEMENTED
Query 7	Configuration Management	IMPLEMENTED
Query 8	Online SOPs	IMPLEMENTED
Query 9	Access Control & Rights Management	IMPLEMENTED

1 How is Internal Data Secured? (ISO Query)

- ▶ **Firestore Authentication:** All data access requires a valid authenticated session. Every Firestore Security Rule begins with `isAuth()` — unauthenticated requests are denied at the database infrastructure level before reaching any data.
- ▶ **Role-Based Access Control (RBAC):** Three enforced roles — `isAdmin()`, `isDoctor()`, `isPatient()` — each with precisely scoped read/write permissions defined in Firestore Security Rules.
- ▶ **Data Ownership Enforcement:** Every record is owner-scoped. A patient cannot read another patient's health records, prescriptions, wallet transactions, or personal data. This is enforced at the database rule level, not application level.
- ▶ **Field-Level Protection:** Sensitive fields such as `role`, `earnings`, `verificationStatus`, and `createdAt` are write-protected via `doesNotModify()` rules — clients cannot tamper with them even if authenticated.
- ▶ **Immutable Audit Logs:** `operation_logs`, `login_audit` events, wallet transactions, and chat messages are strictly append-only. The Firestore rule `update, delete: if false` is applied — no one can alter records after creation.
- ▶ **Payment Security:** Razorpay payment signatures are verified server-side using HMAC-SHA256 in Cloud Functions. Firebase App Check is enforced on both payment endpoints — only verified MedNu app binaries can invoke them.
- ▶ **Firestore App Check:** Activated on both MedNu Patient and MedNu Doctor apps using Play Integrity (Android) and App Attest (iOS) in production. Debug providers used only in development builds.
- ▶ **Encryption in Transit:** All Firebase SDK communication uses TLS 1.2+ by default. No plain-text communication channels are used.
- ▶ **Biometric Lock Screen:** Both apps enforce biometric/MPIN re-authentication when returning from background, preventing unauthorised physical access.
- ▶ **Deny-by-Default:** Firestore Security Rules include an explicit catch-all rule — `match /{document=**} { allow read, write: if false; }` — blocking any collection not explicitly covered by rules.

2 Data Breach & Cyber Fraud — What Steps Are Taken?

- ▶ **Login Audit Trail:** Every login attempt (success and failure) is recorded in the `login_audit` Firestore collection with timestamp, authentication method, and result. Records have a 1-year automated TTL expiry.
- ▶ **Account Lockout:** After 5 consecutive failed MPIN attempts, the account is automatically locked for 30 minutes. Lock state is enforced both client-side and written to Firestore.
- ▶ **MPIN Hashing:** MPIN is stored as `SHA-256(uid:mpin:MEDNU_V1)` — a per-user salted hash. The plain-text MPIN is never stored anywhere in the system.
- ▶ **Operation Logs:** Every significant action (booking, payment, cancellation, profile update) is written to `operation_logs` — append-only, admin-read-only, with a 2-year automated TTL retention policy.
- ▶ **Payment Fraud Prevention:** Razorpay payment signature is verified server-side via HMAC-SHA256 before any order is marked as complete. Firebase App Check blocks unofficial clients from calling payment Cloud Functions.
- ▶ **Emergency Request Throttling:** Emergency doctor requests are deduplicated per patient per 5 minutes — preventing abuse and denial-of-service on the emergency broadcast system.
- ▶ **Serious Complaint Escalation:** A dedicated `serious_complaints` collection triggers an instant Cloud Function that creates an admin alert for immediate human review.
- ▶ **Deny-All Fallback:** The Firestore catch-all rule `allow read, write: if false` ensures any uncovered collection or data access attempt is blocked entirely.
- ▶ **Crashlytics Monitoring:** Firebase Crashlytics monitors all runtime errors, uncaught exceptions, and fatal crashes in real time. User identifiers are attached for incident tracing.

3 How is Information Security Maintained for Cloud Services?

MedNu uses **Google Firebase (Google Cloud Platform)** as its sole cloud infrastructure provider. The following security controls are in place:

- ▶ **Cloud Firestore:** Serverless NoSQL database with fine-grained Security Rules enforced at Google's infrastructure level — not in application code. Rules are version-controlled and deployed via Firebase CLI.
- ▶ **Firebase Authentication:** OTP-based phone number authentication. MedNu never stores raw passwords. Session tokens are managed entirely by Firebase/Google.
- ▶ **Firebase App Check:** Cryptographically verifies that API calls originate from legitimate MedNu app binaries only. Uses Play Integrity (Android) and App Attest (iOS) in production.
- ▶ **Firebase Cloud Functions:** All sensitive operations (payment verification, emergency alerts, weekly quality reports) execute server-side only — the client never has access to business logic or secret keys.
- ▶ **Environment Secrets:** Razorpay API key and secret are stored in Firebase environment variables / Google Secret Manager — never hardcoded in source code or client apps.
- ▶ **Firestore TTL Policies:** Active TTL policies on `operation_logs`, `login_audit/events`, `patient_notifications/items`, and `doctor_notifications/items` — data is automatically deleted after its defined retention period.
- ▶ **Firebase Cloud Messaging (FCM):** Secure push notification delivery over Google's infrastructure. Notifications are device-token gated — no open broadcast endpoints.
- ▶ **Google Cloud Compliance:** Firebase/GCP infrastructure holds ISO 27001, SOC 2 Type II, GDPR, and HIPAA-eligible certifications at the infrastructure level.

4 Access to Source Code

- ▶ **Private Repository:** All source code is hosted in a private GitHub repository under `mednuapp-sketch` organisation. The repository is not publicly accessible.
- ▶ **Role-Based Contributor Access:** Repository access is granted per-developer via GitHub permission levels (Read / Write / Admin). Access is reviewed and revoked when team membership changes.
- ▶ **No Credentials in Code:** Razorpay API keys are in Firebase environment variables. Firebase configuration is in `google-services.json` (Android) and `GoogleService-Info.plist` (iOS), which are access-controlled and not publicly exposed.
- ▶ **Firebase Console Access:** Firestore rules, Cloud Functions, and database configurations are managed via Firebase Console with Google account authentication. Multi-factor authentication is recommended for all console accounts.
- ▶ **Admin Panel Access:** The `mednu-admin` panel is deployed on Firebase Hosting with `isAdmin()` Firestore rule enforced — only verified admin accounts can read or write admin data.
- ▶ **Dependency Security:** An `npm run audit` script is configured in Cloud Functions to detect vulnerable dependencies before every deployment. Runs `npm audit --audit-level=moderate`.
- ▶ **Version Control:** All security-relevant files (Firestore rules, indexes, Cloud Functions) are version-controlled in Git with full change history maintained.

5 Data Storage, Retention & Accidental Deletion Protection

Data Retention Policy (enforced via Firestore TTL):

Data Category	Collection	Retention Period	Mechanism
Operation Audit Logs	operation_logs	2 Years	Firestore TTL (expiresAt)
Login Audit Events	login_audit/events	1 Year	Firestore TTL (expiresAt)
Patient Notifications	patient_notifications/items	90 Days	Firestore TTL (expiresAt)
Doctor Notifications	doctor_notifications/items	90 Days	Firestore TTL (expiresAt)
Medical Records & Prescriptions	health_records, prescriptions	Indefinite	Medical compliance
Payment Records	payments, wallet_transactions	Indefinite	Financial compliance
Appointments	appointments	Indefinite	Medical compliance

Accidental Deletion Protection:

- ▶ **Transaction Immutability:** wallet_transactions and all chat messages have update, delete: if false in Firestore rules — even admins cannot delete them via client SDK.
- ▶ **No Client-Side Mass Delete:** No collection permits bulk deletion from the client. All deletes require admin role and go through the Firebase Console or Cloud Functions.
- ▶ **Account Deletion Workflow:** Patients cannot self-delete. They submit a deletion_request document. Actual deletion is processed by Cloud Functions only — not directly from the client app.
- ▶ **TTL-Only Expiry:** Automated data expiry happens exclusively via Firestore TTL policies configured in Google Cloud Console — not via any client-accessible delete operation.

6 How Web Filtering is Done

- ▶ **Firestore Security Rules (Primary Filter):** Every read and write operation is evaluated against Security Rules before reaching the database. This is enforced at Google's infrastructure level — bypassing it from the client is not possible.
- ▶ **Firebase App Check (API Filter):** Only cryptographically verified MedNu app binaries (Play Integrity on Android, App Attest on iOS) can access any Firebase service. Web scrapers, bots, API abuse tools, and unofficial clients are blocked at the SDK attestation level.
- ▶ **App Check Enforced on Payment Endpoints:** createRazorpayOrder and verifyRazorpayPayment Cloud Functions both have enforceAppCheck: true — any call without a valid App Check token is rejected before the function body executes.
- ▶ **Cloud Function Input Validation:** createRazorpayOrder validates that amount ≥ 100 paise, is of correct numeric type, and all required fields are present. Invalid requests receive `HttpsError('invalid-argument')`.
- ▶ **Notification Deduplication:** The same notification cannot be sent twice within 30 seconds for the same booking — preventing notification spam and flood attacks.
- ▶ **FCM Token-Gated Notifications:** Push notifications are sent only to registered device FCM tokens. There are no open broadcast HTTP endpoints accessible externally.
- ▶ **Admin Panel Domain Restriction:** The admin panel is served exclusively via Firebase Hosting with Firebase Auth enforced. Admin data is only accessible to accounts verified in the `admins` Firestore collection.

7 How and by Whom is Configuration Management Done?

Responsible Party: MedNu Healthcare Services Pvt. Ltd. — development and operations team manages all configuration.

Configuration Item	Location	Managed By	Deployment Method
Firestore Security Rules	firestore.rules (Git)	Dev team	Firebase CLI
Firestore Indexes	firestore.indexes.json (Git)	Dev team	Firebase CLI
TTL Policies	Google Cloud Console	Dev team	Cloud Console UI
Cloud Functions	functions/index.js (Git)	Dev team	Firebase CLI
Function Dependencies	functions/package.json (Git)	Dev team	npm + npm audit
Flutter App Config	firebase_options.dart (Git)	Dev team	Flutter build
Environment Secrets	Firebase Secret Manager	Project owner	Firebase Console
App Check Providers	Firebase Console	Project owner	Console UI
Admin Panel	mednu-admin/ (Git)	Dev team	Firebase Hosting CLI

Version Control: All configuration files are tracked in Git with full change history. Any modification to security rules or Cloud Functions is committed with a descriptive message before deployment.

8 Online SOPs (Standard Operating Procedures)

MedNu has the following **automated SOPs embedded and enforced by the system**:

SOP	Trigger	Automated Action
Emergency Doctor Request	Patient taps Emergency	Broadcasts FCM to all active doctors, creates admin alert, deduplicates within 5 minutes
Weekly Quality Review	Every Monday 8:00 AM IST	Cloud Function aggregates 7-day consultation feedback, ratings, cancellations into quality report
Stale Consultation Expiry	Every 2 minutes (scheduled)	Pending consultations older than 90 seconds are auto-marked as missed
Payment Verification	Every payment completion	Server-side HMAC-SHA256 signature verification before order is confirmed
Account Lockout	5 failed MPIN attempts	Account automatically locked for 30 minutes, written to Firestore
Account Deletion	Patient requests deletion	deletion_request created → admin reviews → Cloud Function processes deletion
Serious Complaint Escalation	Complaint submitted	Cloud Function instantly creates admin_alerts entry for human review
Data Retention Enforcement	Continuous (Firestore TTL)	Logs auto-deleted after 2 years, login audit after 1 year, notifications after 90 days
Rating Aggregation	New review submitted	Atomic Firestore transaction updates doctor rating summary in real time
Login Audit Logging	Every login attempt	Success/failure logged to login_audit with timestamp and 1-year TTL

9 Access Control and Rights — How is it Managed?

MedNu implements a **multi-layer access control system** covering authentication, authorisation, role management, and session security.

Layer 1 — Authentication (Who are you?)

- ▶ **OTP Phone Verification:** Every user must verify their phone number via Firebase Phone Authentication (OTP) before any access is granted. Firebase manages the OTP delivery, verification, and session token issuance.
- ▶ **MPIN (Personal Identification Number):** After OTP, users set a 6-digit MPIN. It is stored as SHA-256(`uid:mpin:MEDNU_V1`) — a per-user salted hash. The plain-text MPIN is never stored anywhere in the system.
- ▶ **Biometric Authentication:** Both MedNu Patient and MedNu Doctor apps support fingerprint/face biometric authentication via the device's secure enclave. Biometric is enforced every time the app returns from background.
- ▶ **Account Lockout Policy:** After 5 consecutive failed MPIN attempts, the account is automatically locked for 15 minutes. The lockout state is written to Firestore and enforced server-side — not just client-side.
- ▶ **Session Management:** Firebase Authentication issues JWT tokens with automatic expiry. Token refresh is handled by the Firebase SDK. Users are required to re-authenticate via OTP if the session expires.

Layer 2 — Authorisation (What can you access?)

- ▶ **Role-Based Access Control (RBAC):** Three distinct roles are enforced in Firestore Security Rules: Admin, Doctor, and Patient. Each role has strictly defined read/write permissions — no role can escalate its own privileges.
- ▶ **Role Assignment:** The role field is set to 'patient' server-side at registration and protected by the Firestore rule `doesNotModify(['role', 'createdAt'])` — a patient cannot change their own role to admin or doctor from the client app.
- ▶ **Admin Rights:** Admin access is granted only by the existence of a document in the `admins` Firestore collection for that user's UID. This collection has `allow write: if false` — only the Firebase Console or Cloud Functions can grant admin rights. No self-promotion is possible.
- ▶ **Doctor Rights:** Doctors must be in the `doctors` collection with `status: 'active'`. Their profile is verified and approved by an admin before they gain doctor-level access rights.
- ▶ **Data Ownership:** Every data record enforces owner-scoped access via `isOwner(uid)`. Patients can only access their own health records, wallet, notifications, prescriptions, and family data.

Layer 3 — Rights Matrix

Resource	Patient (Owner)	Doctor	Admin
Own health records	Read / Write	Read	Read
Other patient records	No Access	Read (assigned)	Read
Own appointments	Read / Create	Read / Update	Full
Prescriptions	Read (own)	Create	Full
Wallet & Transactions	Read (own)	No Access	Read
Doctor profiles	Read (active only)	Read / Update (own)	Full
Admin data	No Access	No Access	Full
Operation logs	Create (own)	Create (own)	Read All
Login audit	Create (own)	No Access	Read All
Banners / Config	Read	Read	Full

Layer 4 — Sensitive Data Protection

- ▶ **Flutter Secure Storage:** Sensitive local data (MPIN hash, last logged-in phone number) is stored using flutter_secure_storage with AndroidOptions(encryptedSharedPreferences: true) — stored in Android Keystore / iOS Keychain, not plain SharedPreferences.
- ▶ **Protected Fields:** Fields like role, createdAt, earnings, verificationStatus, rating are immutable from the client — protected by doesNotModify() in Firestore rules.
- ▶ **Immutable Financial Records:** Wallet transactions are update, delete: if false — no one can alter or delete a financial record after it is created, even admins via the client SDK.
- ▶ **Crashlytics User Tracking:** Firebase Crashlytics tracks the authenticated user UID on all crash reports, enabling accurate incident attribution without exposing personal data.

Summary: Access control in MedNu is enforced at three independent levels — Firebase Authentication (identity), Firestore Security Rules (database), and application-level MPIN/biometric (device). No single layer is relied upon exclusively, providing defence-in-depth consistent with ISO 27001 Annex A.5.15 (Access Control) and A.8.3 (Information Access Restriction).